

8. Classical Problems: Readers-Writers Problem

Syllabus Coverage: Gateway Classes Slides 76–92 | Conflict Matrix, First Readers-Writers Problem, Execution Trace, Writer Starvation.

8.1 Access Rules & Protocols

Readers may read concurrently; Writers require exclusive mutual exclusion.

- `semaphore mutex = 1;` — Protects reader counter `rc`.
- `semaphore db = 1;` — Mutual exclusion lock for the database.

Readers-Writers Problem Architecture

Concurrent Readers, Exclusive Writer & Writer Starvation Mechanics

Conflict & Permission Matrix

Reader & Reader: ✓ ALLOWED (Concurrent)

Reader & Writer: ✗ FORBIDDEN (Conflict)

Writer & Writer: ✗ FORBIDDEN (Conflict)

Key Insight: Only Writers require strict mutual exclusion!

The Semaphore Protocols

READER PROTOCOL (`rc` = reader count):

```
wait(mutex); rc++;
if (rc == 1) wait(db); // 1st reader locks DB for all readers
signal(mutex);
/* READ DATABASE CONCURRENTLY */
wait(mutex); rc--;
if (rc == 0) signal(db); // Last reader unlocks DB
signal(mutex);
```

WRITER PROTOCOL:

```
wait(db); // Exclusively locks DB
/* WRITE DATABASE */
signal(db); // Releases DB
```

⚠ THE PROBLEM OF WRITER STARVATION (First Readers-Writers Problem)

Agar Readers continuously aate rahein (R1 enters, then R2 enters before R1 finishes, then R3 enters...),
toh reader count 'rc' KABHI BHI 0 NAHI HOGA!
 Nateeja: Database lock 'db' kabhi release nahi hoga, aur waiting Writer process hamesha ke liye starvation
 mein phas jayega. Isse solve karne ke liye Fair / Writer-Priority Solution use kiya jata hai.

8.2 The "First In, Last Out" Semaphore Lock Rule

```
// First reader locks DB for all readers
if (rc == 1) wait(db);

// Last reader unlocks DB for writers
if (rc == 0) signal(db);
```

8.3 Starvation Dilemma

If new readers arrive continuously before active readers finish, `rc` never reaches zero, leading to indefinite starvation of pending writers.